

CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 1 Report

Team Members: Shawn Robinson

Jay Patel

Project Teams Group #: IS_01

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.

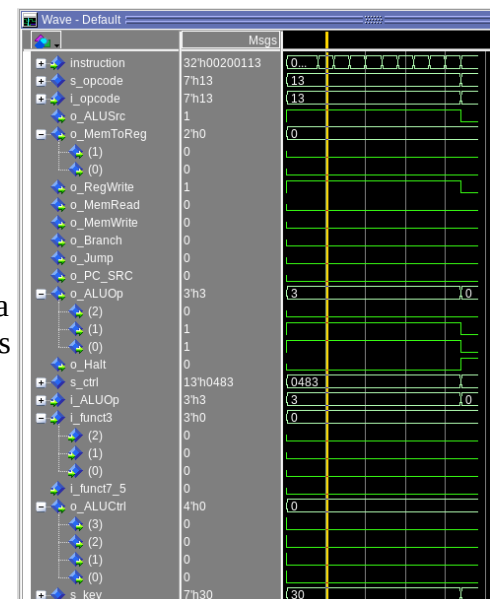
Jay completed a comprehensive digital design with every signal and every component, which can be zoomed in and out of at leisure. I am having trouble getting that to format properly for this .odt file... So, I'm going to provide the link to that design, and then point you toward my finish pencil-paper schematic which is the very last image in this lab report!

[Part 3.1.a.] Create a spreadsheet detailing the list of M instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the N control signals needed by your datapath implementation. The end result should be an $N \times M$ table where each row corresponds to the output of the control logic module for a given instruction.

I believe that this spreadsheet doesn't need to be provided here, right? It is completed and within our project folder!

[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).

Provided to the right are all of the control signals generated from a set of instructions in an addiseq, and these correspond with what is expected on our final spreadsheet.



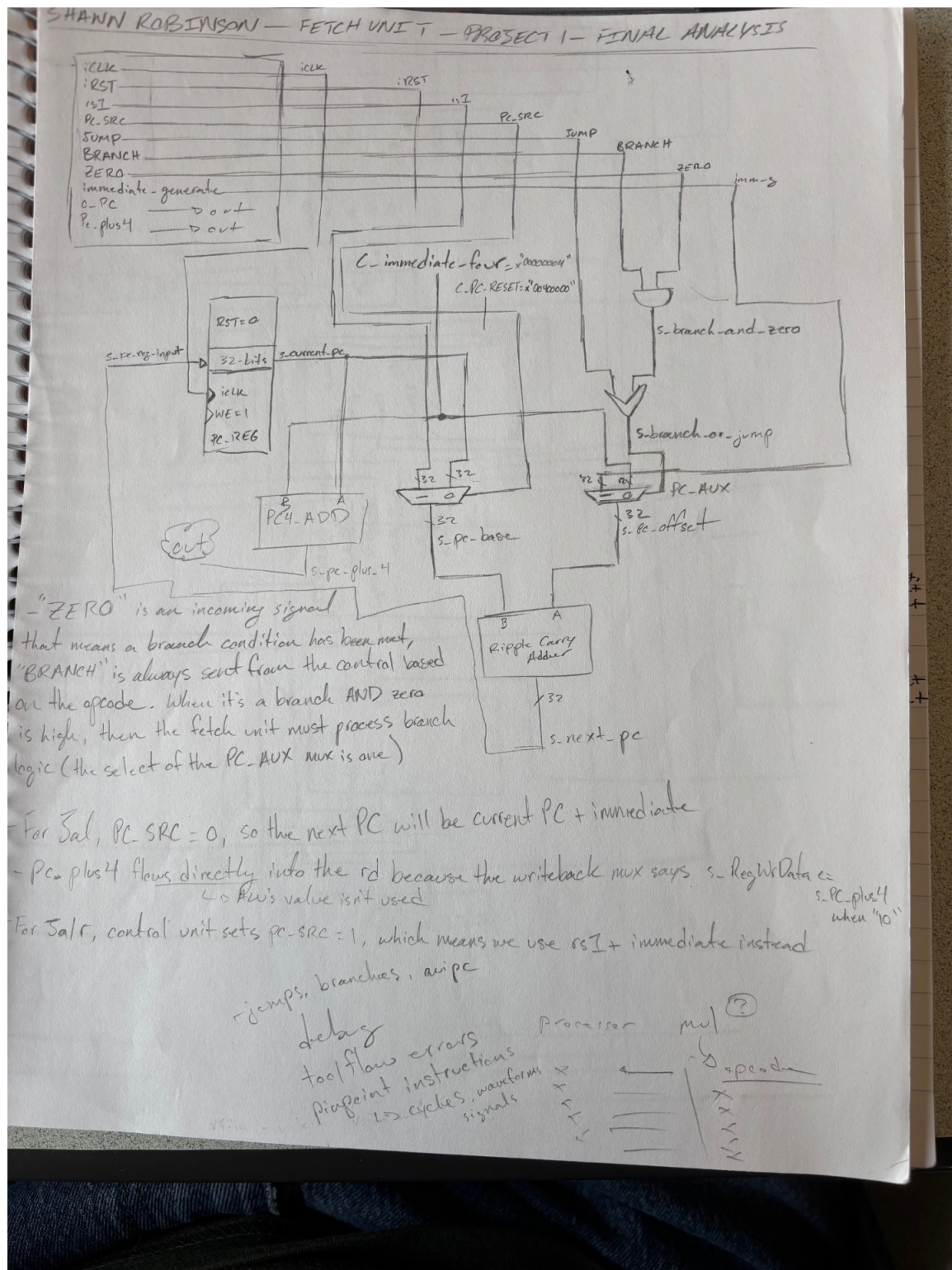
[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

The control flow possibilities that our instruction fetch logic must support are:

- $PC + 4$
- $PC + IMMEDIATE$
- $RS1 + IMMEDIATE$

-Which encompasses whether or not a branch has been taken, whether or not a jump has happened, or whether the PC is continuing forward as would be expected.

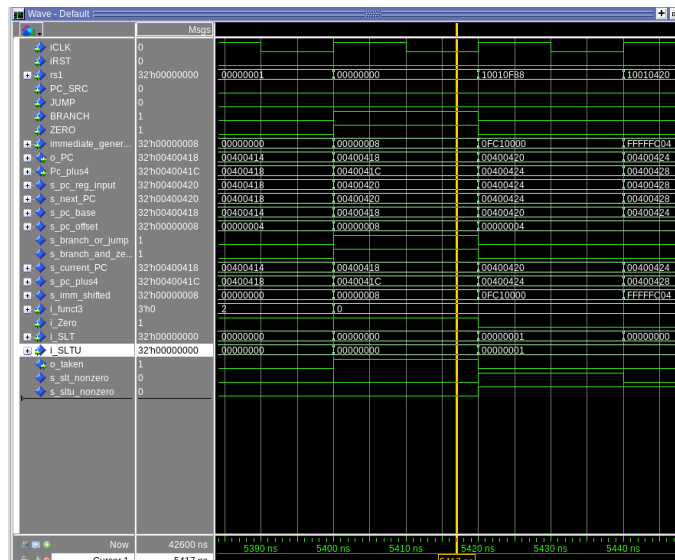
[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?



[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.

Here is an example of the instruction fetch unit working in combination with our “branch resolver” unit on a segment of grendel.s

This screenshot demonstrates the full capacity and robustness of the integrated fetch unit, operating on a SLTU signal produced from the ALU.



[Part 3.3.1.(a)] Describe the difference between logical (srl) and arithmetic (sra) shifts. Why does RISC-V not have a sla instruction?

The logical shift right fills with zeroes, while the arithmetic shift right fills with the sign bit. There isn't an sla because it would actually just do the same thing that sll does already, anyway.

[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.

Our VHDL implements arithmetic and logical shifting by using a general purpose shifter that is fed bits 4:0 of i_B, all of i_A and two raw signals for whether or not it's an arithmetic or left shift. The internal shifter then performs the correct shift depending on these two bits of context. In the ALU these raw signals are hardcoded with the necessary ones or zeros to be “left shifter” or the two types of “right shifter”

[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations.

You can reverse the bits of data, and then use a MUX as a control signal for which one you feed to the shifter.

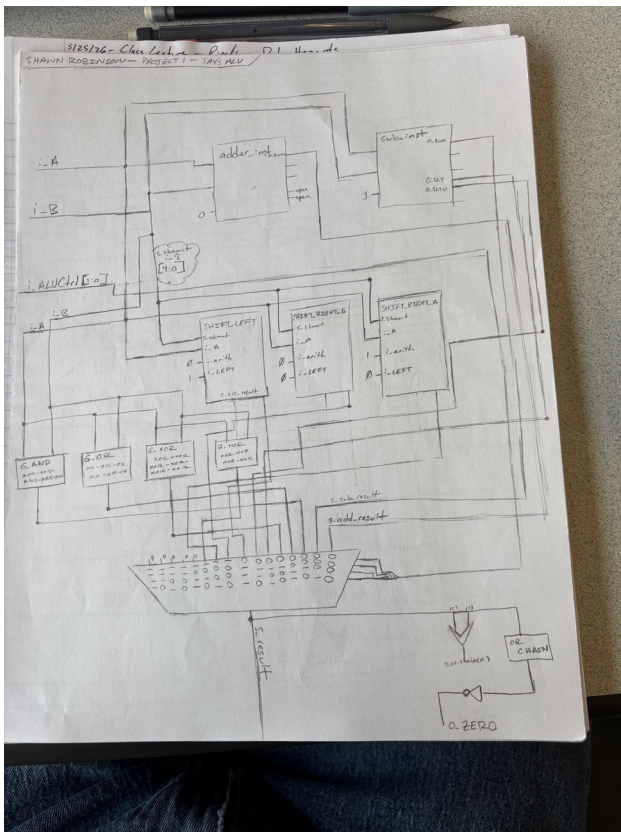
[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.

The outputs of the different shifting operations (shift_left, shift_right_l, and shift_right_A) can be seen in the waveforms below for our ALU test.

Jay designed the ALU to use two adder components, three shifters, and then dedicated and, or, xor, and nor logic components. The lines of i_A and i_B are fed into the adder (and subber), as well as the other pure logic units, and then i_A is fed into the shifters. All the results are put into a 16:1 mux with a 32-bit final output called s_result.

Wave - Default -		Msgs	
+	instruction	32'h00200113	0...
+	immediate_gener...	32'h00000002	0...
+	s_opcode	7h13	13
+	i_A	32'h00000000	00000000
+	i_B	32'h00000002	0...
+	i_ALUCtrl	4h0	0
+	o_Result	32'h00000002	0...
+	o_Zero	0	0
+	o_SLT	32'h00000001	00000001
+	o_SLTU	32'h00000001	00000001
+	s_add_result	32'h00000002	0...
+	s_sub_result	32'hFFFFFFFE	F...
+	s_slt_result	32'h00000001	00000001
+	s_sltu_result	32'h00000001	00000001
+	s_sll_result	32'h00000000	00000000
+	s_srl_result	32'h00000000	00000000
+	s_sra_result	32'h00000000	00000000
+	s_and_result	32'h00000000	00000000
+	s_or_result	32'h00000002	0...
+	s_xor_result	32'h00000002	0...
+	s_Cout_add	0	0
+	s_Cout_sub	0	0
+	s_Ovf_add	0	0
+	s_Ovf_sub	0	0
+	s_shamt	5h02	01
+	s_result	32'h000000002	0...
+	s_or_chain	31'h7FFFFFFF	7FFFFFFF... 7FFF... 7...

[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?

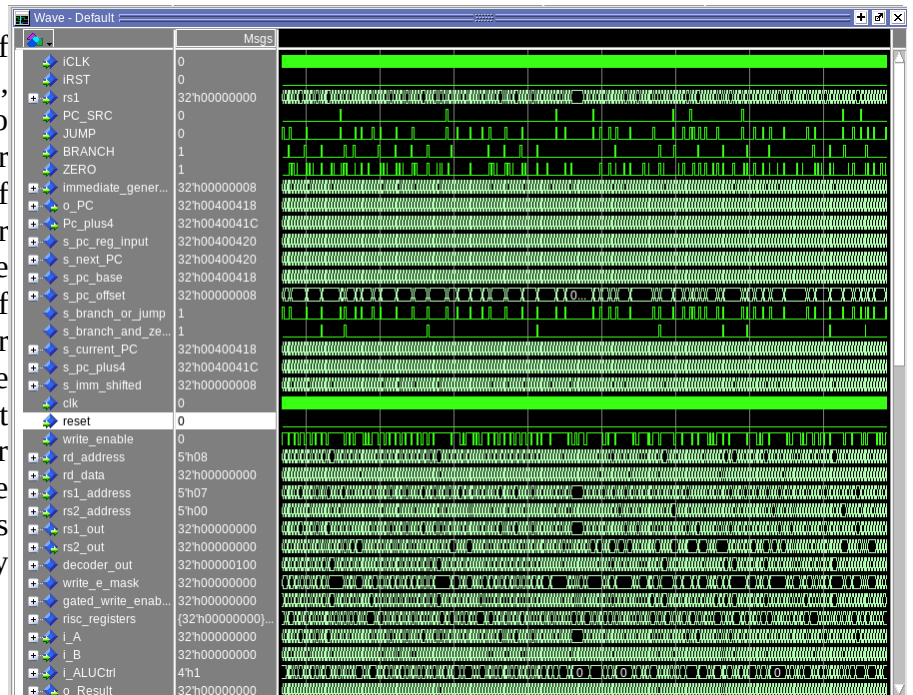


[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

The picture on the previous page shows all the components just described functioning a testbench addi sequence. Each separate response signal is visible and corresponds to accurate output.

[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.

We took the approach of preparing fully-capable units, which we then took the time to integrate together comprehensively. Because of this, the question of how our test plan would be comprehensive kind of answered itself. Our comprehensive tests were the same ones that would be used at the end of this lab to assess our full processor. In that spirit, the waveform I'm providing is relevant waves successfully passing grendel.s



[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.

```

3810.tf.sh cpre3810-toolflow.pdf internal output proj temp
[shawn@co2050-07 cpre3810-toolflow]$ ./3810.tf.sh test --summary proj/riscv/cpre3810_test_assembly_program/*.s
Using LAB Python Environment
Testing
None
All VHDL src files compiled successfully

```

Assembly file	Rars	Questasim	Test Passed	CPI	Results Directory
est_assembly_program/add_1.s	pass	pass	pass	1.0	output/add_1.s
est_assembly_program/add_2.s	pass	pass	pass	1.0	output/add_2.s
est_assembly_program/add_3.s	pass	pass	pass	1.0	output/add_3.s
est_assembly_program/add11.s	pass	pass	pass	1.0	output/add11.s
est_assembly_program/add12.s	pass	pass	pass	1.0	output/add12.s
est_assembly_program/add13.s	pass	pass	pass	1.0	output/add13.s
t_assembly_program/add1_01.s	pass	pass	pass	1.0	output/add1_01.s
t_assembly_program/add1_02.s	pass	pass	pass	1.0	output/add1_02.s
est_assembly_program/and_0.s	pass	pass	pass	1.0	output/and_0.s
est_assembly_program/and_1.s	pass	pass	pass	1.0	output/and_1.s
est_assembly_program/and_2.s	pass	pass	pass	1.0	output/and_2.s
est_assembly_program/and_3.s	pass	pass	pass	1.0	output/and_3.s
est_assembly_program/and11.s	pass	pass	pass	1.0	output/and11.s
est_assembly_program/and12.s	pass	pass	pass	1.0	output/and12.s
est_assembly_program/and13.s	pass	pass	pass	1.0	output/and13.s
t_assembly_program/auipc_1.s	pass	pass	pass	1.0	output/auipc_1.s
t_assembly_program/auipc_2.s	pass	pass	pass	1.0	output/auipc_2.s
t_assembly_program/auipc_3.s	pass	pass	pass	1.0	output/auipc_3.s
est_assembly_program/beq_1.s	pass	pass	pass	0.917	output/beq_1.s
est_assembly_program/beq_2.s	pass	pass	pass	1.0	output/beq_2.s
est_assembly_program/beq_3.s	pass	pass	pass	1.0	output/beq_3.s
est_assembly_program/bge_1.s	pass	pass	pass	1.0	output/bge_1.s
est_assembly_program/bge_2.s	pass	pass	pass	1.0	output/bge_2.s
est_assembly_program/bge_3.s	pass	pass	pass	1.0	output/bge_3.s
st_assembly_program/bgeu_1.s	pass	pass	pass	1.0	output/bgeu_1.s
st_assembly_program/bgeu_2.s	pass	pass	pass	1.0	output/bgeu_2.s
st_assembly_program/bgeu_3.s	pass	pass	pass	1.0	output/bgeu_3.s
test_assembly_program/blt1.s	pass	pass	pass	1.0	output/blt1.s
test_assembly_program/blt2.s	pass	pass	pass	1.0	output/blt2.s
test_assembly_program/blt3.s	pass	pass	pass	1.0	output/blt3.s
ssembly_program/bltu_test1.s	pass	pass	pass	0.769	output/bltu_test1.s
ssembly_program/bltu_test2.s	pass	pass	pass	0.75	output/bltu_test2.s
ssembly_program/bltu_test3.s	pass	pass	pass	0.769	output/bltu_test3.s
est_assembly_program/bne_1.s	pass	pass	pass	1.0	output/bne_1.s
est_assembly_program/bne_2.s	pass	pass	pass	1.0	output/bne_2.s
est_assembly_program/bne_3.s	pass	pass	pass	1.0	output/bne_3.s
est_assembly_program/jal_1.s	pass	pass	pass	1.0	output/jal_1.s
est_assembly_program/jal_2.s	pass	pass	pass	1.0	output/jal_2.s
est_assembly_program/jal_3.s	pass	pass	pass	1.0	output/jal_3.s
st_assembly_program/jalr_1.s	pass	pass	pass	1.0	output/jalr_1.s
st_assembly_program/jalr_2.s	pass	pass	pass	1.0	output/jalr_2.s
st_assembly_program/jalr_3.s	pass	pass	pass	1.0	output/jalr_3.s
test_assembly_program/lb_0.s	pass	pass	pass	1.0	output/lb_0.s
test_assembly_program/lb_1.s	pass	pass	pass	1.0	output/lb_1.s
test_assembly_program/lb_2.s	pass	pass	pass	1.0	output/lb_2.s
est_assembly_program/lbu_1.s	pass	pass	pass	1.0	output/lbu_1.s
est_assembly_program/lbu_2.s	pass	pass	pass	1.0	output/lbu_2.s
est_assembly_program/lbu_3.s	pass	pass	pass	1.0	output/lbu_3.s
test_assembly_program/lh_1.s	pass	pass	pass	1.0	output/lh_1.s
test_assembly_program/lh_2.s	pass	pass	pass	1.0	output/lh_2.s
test_assembly_program/lh_3.s	pass	pass	pass	1.0	output/lh_3.s
est_assembly_program/lhu_1.s	pass	pass	pass	1.0	output/lhu_1.s
est_assembly_program/lhu_2.s	pass	pass	pass	1.0	output/lhu_2.s
est_assembly_program/lhu_3.s	pass	pass	pass	1.0	output/lhu_3.s
est_assembly_program/lui_1.s	pass	pass	pass	1.0	output/lui_1.s
est_assembly_program/lui_2.s	pass	pass	pass	1.0	output/lui_2.s
est_assembly_program/lui_3.s	pass	pass	pass	1.0	output/lui_3.s
test_assembly_program/lw_0.s	pass	pass	pass	1.0	output/lw_0.s
test_assembly_program/lw_1.s	pass	pass	pass	1.0	output/lw_1.s
test_assembly_program/lw_2.s	pass	pass	pass	1.0	output/lw_2.s
test_assembly_program/or_1.s	pass	pass	pass	1.0	output/or_1.s
test_assembly_program/or_2.s	pass	pass	pass	1.0	output/or_2.s
test_assembly_program/or_3.s	pass	pass	pass	1.0	output/or_3.s
est_assembly_program/ori_1.s	pass	pass	pass	1.0	output/ori_1.s
est_assembly_program/ori_2.s	pass	pass	pass	1.0	output/ori_2.s
est_assembly_program/ori_3.s	pass	pass	pass	1.0	output/ori_3.s
est_assembly_program/sll_1.s	pass	pass	pass	1.0	output/sll_1.s
est_assembly_program/sll_2.s	pass	pass	pass	1.0	output/sll_2.s
est_assembly_program/sll_3.s	pass	pass	pass	1.0	output/sll_3.s
st_assembly_program/slli_0.s	pass	pass	pass	1.0	output/slli_0.s
st_assembly_program/slli_1.s	pass	pass	pass	1.0	output/slli_1.s
st_assembly_program/slli_2.s	pass	pass	pass	1.0	output/slli_2.s
test_assembly_program/slt1.s	pass	pass	pass	1.0	output/slt1.s
test_assembly_program/slt2.s	pass	pass	pass	1.0	output/slt2.s
est_assembly_program/slt_1.s	pass	pass	pass	1.0	output/slt_1.s
est_assembly_program/slt_2.s	pass	pass	pass	1.0	output/slt_2.s
est_assembly_program/slt_3.s	pass	pass	pass	1.0	output/slt_3.s
st_assembly_program/slti_1.s	pass	pass	pass	1.0	output/slti_1.s
st_assembly_program/slti_2.s	pass	pass	pass	1.0	output/slti_2.s
st_assembly_program/slti_3.s	pass	pass	pass	1.0	output/slti_3.s
t_assembly_program/sltiu_1.s	pass	pass	pass	1.0	output/sltiu_1.s
t_assembly_program/sltiu_2.s	pass	pass	pass	1.0	output/sltiu_2.s
t_assembly_program/sltiu_3.s	pass	pass	pass	1.0	output/sltiu_3.s
est_assembly_program/sra_1.s	pass	pass	pass	1.0	output/sra_1.s
est_assembly_program/sra_2.s	pass	pass	pass	1.0	output/sra_2.s
est_assembly_program/sra_3.s	pass	pass	pass	1.0	output/sra_3.s
est_assembly_program/sra_4.s	pass	pass	pass	1.0	output/sra_4.s
est_assembly_program/sra_5.s	pass	pass	pass	1.0	output/sra_5.s
st_assembly_program/srai_1.s	pass	pass	pass	1.0	output/srai_1.s
st_assembly_program/srai_2.s	pass	pass	pass	1.0	output/srai_2.s
st_assembly_program/srai_3.s	pass	pass	pass	1.0	output/srai_3.s
test_assembly_program/srl1.s	pass	pass	pass	1.0	output/srl1.s
test_assembly_program/srl2.s	pass	pass	pass	1.0	output/srl2.s
est_assembly_program/srl_1.s	pass	pass	pass	1.0	output/srl_1.s
est_assembly_program/srl_2.s	pass	pass	pass	1.0	output/srl_2.s
est_assembly_program/srl_3.s	pass	pass	pass	1.0	output/srl_3.s
st_assembly_program/srli_1.s	pass	pass	pass	1.0	output/srli_1.s
st_assembly_program/srli_2.s	pass	pass	pass	1.0	output/srli_2.s
st_assembly_program/srli_3.s	pass	pass	pass	1.0	output/srli_3.s
st_assembly_program/srli_4.s	pass	pass	pass	1.0	output/srli_4.s
st_assembly_program/srli_5.s	pass	pass	pass	1.0	output/srli_5.s
est_assembly_program/sub_0.s	pass	pass	pass	1.0	output/sub_0.s
st_assembly_program/sub_00.s	pass	pass	pass	1.0	output/sub_00.s
st_assembly_program/sub_01.s	pass	pass	pass	1.0	output/sub_01.s
st_assembly_program/sub_02.s	pass	pass	pass	1.0	output/sub_02.s
est_assembly_program/sub_1.s	pass	pass	pass	1.0	output/sub_1.s
est_assembly_program/sub_3.s	pass	pass	pass	1.0	output/sub_3.s
test_assembly_program/sw_0.s	pass	pass	pass	1.0	output/sw_0.s
test_assembly_program/sw_1.s	pass	pass	pass	1.0	output/sw_1.s
test_assembly_program/sw_2.s	pass	pass	pass	1.0	output/sw_2.s
est_assembly_program/xor_0.s	pass	pass	pass	1.0	output/xor_0.s
est_assembly_program/xor_1.s	pass	pass	pass	1.0	output/xor_1.s
est_assembly_program/xor_2.s	pass	pass	pass	1.0	output/xor_2.s
st_assembly_program/xori_1.s	pass	pass	pass	1.0	output/xori_1.s
st_assembly_program/xori_2.s	pass	pass	pass	1.0	output/xori_2.s
st_assembly_program/xori_3.s	pass	pass	pass	1.0	output/xori_3.s

```

[shawn@co2050-07 cpre3810-toolflow]$

```

[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.

[Part 4.c] Create and test an application that sorts an array with N elements using the MergeSort algorithm ([link](#)). Name this file Proj1_mergesort.s.

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

The maximum frequency our processor can run at currently (without making the changes we will discuss below) is 19.2 MHz. ($1 / (52 \text{ ns} * 10^{-9})$)

Our critical path actually falls on branching, not on loading, due to a component we added called “branch resolver”, which is in series with the ALU instead of being in parallel back around the control unit. The ripple carry adders in our fetch_unit and alu_32 eat up ~13 ns each, meaning that over half our total execution time is taken up just rippling through these additions.

The obvious place for improvement will be the implementation of carry lookahead adders, as well as changing the placement of the branch resolver lines.

